ClearCut: Spoiler-Free Rewriting of Movie and TV Reviews **Code:** https://github.com/tejeshwar20/clearcut

1. Abstract

Movie and TV reviews often reveal twists, deaths and endings. ClearCut is a web application that takes pasted English reviews and returns a spoiler-free version, with a short aspect-based verdict. Only sentences that disclose plot are rewritten. All other sentences are kept exactly as the reviewer wrote them. The system is a rule-based NLP pipeline that runs locally with no API key.

2. Introduction

Readers want a reviewer's opinion (acting, story, pacing, visuals) without learning what happens. Existing "spoiler tags" depend on the reviewer, and simply deleting a flagged sentence can drop the reviewer's opinion with it. ClearCut keeps the opinion and hides only the disclosed fact.

3. Problem Statement

Online reviews are the main way people decide what to watch, but many of them reveal plot twists, character deaths and endings. Spoiler tags are optional and depend on the reviewer, and many reviews have none. Deleting a whole flagged review, or a whole flagged sentence, also throws away the reviewer's opinion, which is the part the reader actually wants. Readers are left choosing between being spoiled and not reading reviews at all.

Problem: given one or more raw English reviews of a movie or TV show, automatically produce a version that 1. contains **no plot disclosures** (identity reveals, deaths, twists, endings); 2. **keeps the reviewer's opinion** and leaves harmless sentences exactly as written; 3. gives a short, readable **verdict by aspect** (overall impression, acting, story and writing, pacing, visuals, characters); and 4. **fails safe**: if a disclosure cannot be removed with confidence, the text is withheld rather than shown, because a leaked spoiler is the worst failure.

Challenges: spoilers are expressed in many ways ("Darth Vader is Luke's father", "it was all a simulation", "he doesn't make it") and usually share no keyword with the harmless text around them. Removing too much destroys the review, and removing too little leaks the plot. No large labelled spoiler dataset is available for this kind of rewriting, so the system must work without one and be measured honestly.

4. Solution Approach

The system is a pipeline in `pipeline.py` and `source_writer.py`:

1. **Preprocessing:** input validation, Unicode normalisation, and sentence splitting with NLTK Punkt.
2. **Spoiler detection:** curated cue patterns for identity reveals, deaths and fates, "the whole time" twists, dream or simulation twists, betrayal, faked events, endings and explicit "Spoiler:" markers. These are combined with a **spaCy dependency parse** (`en_core_web_sm`) to decide how much of a sentence to remove.
3. **Rewriting:** the reviewer's reaction is kept and the revealed fact is replaced with a neutral phrase. For example, "In the end the hero dies saving his village" becomes "The ending is best experienced without knowing what's coming."
4. **Sentence selection:** TF-IDF and cosine similarity choose diverse, non-redundant sentences when several reviews are combined.
5. **Final safety check:** the output is re-scanned and withheld if any disclosure remains.

Stack: Python, Flask, spaCy, NLTK, scikit-learn, VADER, Waitress, and a plain HTML/JS front end.

5. Experimental Results

Tests were run with `artifacts/measure.py`.

Measure	Result
Spoiler sentences leaked	**0 of 25**
Harmless sentences changed or blocked	**0 of 20**
Mixed reviews with surrounding harmless sentences intact	**48 of 48**
Latency, short review (median / 95th percentile)	20 ms / 26 ms
Latency, 297-word review (median / max)	127 ms / 170 ms
Automated tests passing	212 of 212

6. Data Source

- **User-supplied reviews:** the text pasted into the page (`data/sample_reviews.csv` is a sample). Reviews are processed in memory and are not stored.
- **Evaluation data:** `data/evaluation_examples.json` (26 synthetic labelled sentences) and the general cases in `artifacts/measure.py`.
- **Pretrained models:** spaCy `en_core_web_sm` and the VADER lexicon.

7. Project Files

File	Purpose
`app.py`	Flask web app: serves the page and the review-analysis API
`pipeline.py`	Core NLP: spoiler detection, rewriting, aspect sentiment, sentence selection, safety check
`source_writer.py`	Builds the reader-facing text from opinions using dependency parsing
`editorial_writer.py`	Optional LLM polish of spoiler-free text (off by default)
`movie_source.py`, `movie_store.py`	Optional title lookup and storage of saved reports (not used by the UI)
`serve.py`	Production entry point (Waitress)
`evaluate.py`	Scores aspect sentiment on `data/evaluation_examples.json`
`templates/index.html`, `static/`	Front end: page, JavaScript, CSS, logo
`data/`	Sample reviews, evaluation examples, catalog files
`tests/`	212 automated tests
`artifacts/`	Measurement and spoiler-probe scripts (`measure.py`, `probe_spoilers.py`)
`requirements.txt`, `.env.example`	Dependencies and settings template